

DETECTION/ML HANDOFF

Detection -> Full-Stack Integration Guide

ThreatAlert v1.1 API contract, storage/WebSocket flow, UI semantics, and integration pitfalls

Audience	Backend / database / WebSocket / dashboard team
Status	Integration handoff - do not redesign Detection
Baseline	Detection baseline: 254fc16 (verified)

Purpose This document defines the exact boundary between the finished Detection/ML module and your module. Follow it as a contract: preserve raw evidence, do not invent values, and do not move detection logic across the boundary.

SIH 2026 - AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

1. Integration objective

Full-stack does not run the detection models. It receives already-standardized ThreatAlert v1.1 objects, persists them, broadcasts them, and renders them without changing their meaning. Detection owns threat classification, score, severity, scope and evidence.

Detection 7 detectors + DGA ML	ThreatAlert v1.1 frozen JSON object	POST /api/v1/alerts	Backend validate + persist	PostgreSQL alert record + evidence JSON	WebSocket /ws/alerts -> dashboard
---	---	-------------------------------	--------------------------------------	--	--

Key rule Treat ThreatAlert v1.1 as a wire contract, not a suggestion. Do not recompute severity, reinterpret score as probability, or invent missing aggregate fields.

2. ThreatAlert v1.1 - exact wire fields

Field	Type	Full-stack handling
alert_id	UUID v4 string	Primary external identity. Preserve exactly; recommended idempotency key.
schema_version	literal "1.1"	Validate and store. Reject/route unsupported versions explicitly.
event_start	ISO-8601 UTC	Event window start, literal Z on wire.
event_end	ISO-8601 UTC	Must be >= event_start.
detected_at	ISO-8601 UTC	When Detection emitted the alert.
event_scope	enum	flow / source_host / destination_host / host_pair / network.
flow_id	string or null	Null is normal for aggregate alerts.
src_ip / dst_ip	string or null	May be null when no single truthful value represents an aggregate.
dst_port	integer 0-65535 or null	Null is valid. Do not convert null to 0.
protocol	string or null	Null is valid for mixed/aggregate contexts.
threat_class	enum	One of seven frozen classes.
severity	enum	low / medium / high / critical. Display as supplied.
score	float 0.0-1.0	UI label: Threat Score. Not automatically a probability.
score_type	enum	calibrated_model / rule_score / anomaly_score / signature_match.
evidence	JSON object	Preserve as structured JSON. Shape varies by detector/version.
detector	string	Producer name.
detector_version	string	Producer version.

Field	Type	Full-stack handling
mitre_techniques	array[string]	Technique IDs such as T1046.
incident_id	string or null	Detection currently leaves it null; backend may correlate later.

3. Frozen enum vocabularies

Enum	Allowed values
threat_class	dga_domain, dns_tunnelling, c2_beaconing, encrypted_malware, ddos, port_scan, data_exfiltration
severity	low, medium, high, critical
score_type	calibrated_model, rule_score, anomaly_score, signature_match
event_scope	flow, source_host, destination_host, host_pair, network

Schema versioning Do not add/rename/remove enum members in the API without a ThreatAlert schema-version change. Evidence may evolve; the top-level contract is frozen at 1.1.

4. Score semantics - the most important UI rule

The normalized top-level score is always finite and between 0 and 1, but its meaning depends on score_type. The dashboard should call it "Threat Score", not "Probability".

score_type	Meaning	Recommended UI treatment
signature_match	Exact configured IOC/signature match path.	Show Threat Score + "Signature match" badge.
rule_score	Normalized score derived from deterministic rule strength / margin.	Show Threat Score + "Rule score" badge. Do not call it model confidence.
anomaly_score	Normalized anomaly strength.	Show Threat Score + "Anomaly" badge.
calibrated_model	A calibrated model probability-like score when a detector truly provides one.	Only here may probability wording be appropriate, and only if the detector documentation says so.

DGA example DGA publishes score_type=rule_score. Its raw Random Forest vote is kept separately in evidence as dga_model_score with model_score_is_calibrated=false. Do not relabel the top-level score or raw RF vote as probability.

5. Aggregate alert semantics and nullable fields

Several detectors report behavior spanning many flows. A single `flow_id`, destination, port or protocol may not truthfully represent the alert. Null is the correct value in that case.

Threat class	Typical scope	What the UI should anchor on
port_scan	source_host	src_ip; many ports/hosts may be summarized in evidence.
ddos	destination_host	dst_ip; src_ip may be null because many sources contributed.
c2_beaconing	host_pair	src_ip + dst_ip + dst_port/protocol when truthful.
dns_tunnelling	host_pair	source + resolver/peer; aggregate DNS evidence.
data_exfiltration	host_pair	src_ip + dst_ip; orig-side volume is key evidence.
encrypted_malware	flow for signature; host_pair for heuristic	Fingerprint match may be a single flow; SNI heuristic can span a pair.
dga_domain	source_host	src_ip; multiple DGA-positive domains are correlated to avoid alert storms.

Never fake aggregates Do not replace null with "multiple", "N/A" inside the stored field, 0, or a placeholder IP. If the UI wants a label such as "Multiple destinations", derive it for display only and keep the stored contract value null.

6. Recommended backend API behavior

1. Expose POST `/api/v1/alerts` and accept exactly one ThreatAlert v1.1 JSON object per request.
2. Validate `schema_version` and required types. Preserve evidence as JSON, not a flattened string.
3. Persist the alert before broadcasting it so database and WebSocket views do not diverge.
4. Return a normal 2xx response only after the backend has accepted the alert. Use stable 4xx for malformed/unsupported schema and 5xx for backend failure.
5. Recommended: make `alert_id` idempotent/unique so a replayed POST does not create duplicate rows.
6. After persistence, broadcast the canonical stored alert on `/ws/alerts` to connected dashboards.
7. Do not make Detection depend on WebSocket availability; Detection only POSTs alerts.

Delivery behavior from Detection If backend POSTs fail, Detection continues processing the capture, counts delivery failures, preserves local JSONL output, and exits with code 2 (degraded). The backend should therefore be safe to receive replays.

7. Example ThreatAlert v1.1 POST

```
{
  "alert_id": "123e4567-e89b-42d3-a456-426614174000",
  "schema_version": "1.1",
  "event_start": "2026-08-30T00:10:00Z",
  "event_end": "2026-08-30T00:10:50Z",
  "detected_at": "2026-08-30T00:10:51Z",
  "event_scope": "source_host",
  "flow_id": null,
  "src_ip": "10.0.0.20",
  "dst_ip": null,
  "dst_port": null,
  "protocol": null,
  "threat_class": "dga_domain",
  "severity": "critical",
  "score": 0.90,
  "score_type": "rule_score",
  "evidence": {
    "domain": "kq3v9x2mzt7wp00.com",
    "dga_model_score": 0.95,
    "score_threshold": 0.75,
    "model_score_is_calibrated": false,
    "distinct_dga_domain_count": 6,
    "sample_domains": ["kq3v9x2mzt7wp00.com", "kq3v9x2mzt7wp01.com"]
  },
  "detector": "dga_domain",
  "detector_version": "0.1.0",
  "mitre_techniques": ["T1568.002"],
  "incident_id": null
}
```

8. Database mapping recommendation

Keep first-class searchable columns for stable top-level fields, and a JSON/JSONB column for detector-specific evidence. This prevents schema churn whenever evidence gains a new key.

DB column	Suggested representation	Notes
alert_id	UUID UNIQUE / PRIMARY KEY	Recommended idempotency anchor.
schema_version	text	Store for migration/debugging.
event_start/event_end/detected_at	timestampz	UTC-aware.
event_scope	text/enum	Validate allowed value.
flow_id	text nullable	Do not require.
src_ip/dst_ip	inet nullable	If DB supports inet; otherwise text.
dst_port	integer nullable	Do not default to 0.
protocol	text nullable	
threat_class	text/enum	Index for dashboard filters.
severity	text/enum	Index/filter.

DB column	Suggested representation	Notes
score	double/real	Constrain 0..1.
score_type	text/enum	Keep semantics visible.
evidence	jsonb	Preserve structure exactly.
detector / detector_version	text	Auditability.
mitre_techniques	jsonb/text[]	Multiple IDs possible.
incident_id	text/uuid nullable	Backend correlation may fill later.

9. Dashboard rendering guidance

- **Primary label:** human-readable threat class (e.g. "Port Scan", "DGA Domain").
- **Threat Score:** show 0-1 or percentage visualization, but label it Threat Score; also show score_type.
- **Severity:** render the detector-supplied low/medium/high/critical value. Do not derive severity again from score.
- **Scope:** show whether this is a flow, source host, destination host, host pair or network event.
- **Evidence:** render common keys nicely but always support unknown/new evidence keys generically.
- **MITRE:** show technique IDs as analyst context.
- **Null network fields:** display a UI-friendly dash/"multiple" label only at presentation time, never by mutating stored data.
- **Time:** render event_start-event_end separately from detected_at so analysts can distinguish when activity occurred from when it was detected.

10. Earlier integration pitfalls and corrected rules

Pitfall / earlier risk	Correct behavior now
Calling every score a probability	Use "Threat Score" and display score_type. DGA specifically is rule_score.
Assuming every alert maps to one flow	Respect event_scope. Aggregate alerts legitimately have flow_id=null.
Forcing dst_ip/dst_port/protocol to be present	Null is valid when multiple values contributed.
Using placeholders such as dst_ip="multiple" or dst_port=0	Never mutate the contract. Keep null; format a display label separately.
Recomputing severity in backend/UI	Severity is detector-owned. Store and display as supplied.
Expecting incident_id from Detection	Detection currently sends null. Incident correlation is backend responsibility.
Flattening evidence into a text column	Store structured JSON/JSONB so detector-specific details remain queryable.
Rejecting an alert because evidence contains unfamiliar keys	Evidence is intentionally flexible; only top-level ThreatAlert fields are frozen.
Treating WebSocket as Detection responsibility	Backend owns persistence + WebSocket broadcast. Detection only POSTs.
Dropping alerts when backend is temporarily unavailable	Detection can preserve local JSONL and report degraded delivery; backend should support safe replay/idempotency.

11. Backend/WebSocket acceptance tests

- [] POST a valid v1.1 alert -> 2xx, one DB row, one WebSocket broadcast.
- [] POST the same alert_id twice -> no duplicate row (recommended idempotency behavior).
- [] Unknown schema_version -> clear unsupported-version response, not silent coercion.
- [] Aggregate DGA alert with flow_id/dst_ip/dst_port/protocol null -> accepted and rendered.
- [] DDoS alert with src_ip null -> accepted and rendered.
- [] evidence with detector-specific/new keys -> stored without schema failure.
- [] score outside 0..1 or NaN-like invalid JSON -> rejected.
- [] event_end before event_start -> rejected.
- [] Unknown top-level field -> reject if backend mirrors v1.1 strictness, or at minimum log schema drift; do not silently reinterpret.
- [] WebSocket client reconnects without affecting POST ingestion.
- [] Database failure -> POST fails clearly; no false 2xx.
- [] Backend recovery + alert replay does not duplicate stored alerts.

12. What full-stack must not promise

- No payload decryption - TLS or QUIC.
- No QUIC-specific fingerprint extraction yet.
- No guarantee that a DGA raw model score is calibrated probability.
- No empirical false-positive rate until real benign capture is measured.
- No automatic threat-feed download; JA3/JA4 feeds are local operational inputs.
- No incident correlation from Detection; backend may add this as a separate layer.

13. Definition of done for full-stack integration

- [] POST /api/v1/alerts accepts one exact ThreatAlert v1.1 object.
- [] Every stable top-level field is persisted without reinterpretation.
- [] evidence is stored as structured JSON.
- [] Nullable aggregate fields are accepted and UI-safe.
- [] Threat Score and score_type are shown correctly; no blanket probability wording.
- [] Severity is displayed as sent, not recomputed.
- [] alert_id is unique/idempotent for replay safety.
- [] /ws/alerts broadcasts persisted alerts to the dashboard.
- [] Backend failure returns a real failure status so Detection can report degraded delivery.
- [] Dashboard can filter by threat_class, severity, score, time and detector.
- [] Unknown evidence keys do not break the UI.
- [] No code path assumes incident_id is already populated.

Final integration principle Keep the stable top level schema-strict and keep evidence flexible. This preserves contract safety without coupling the UI to detector internals.